Generalized LL (GLL) Parser

Published: Jul 2, 2022

TLDR; This tutorial is a complete implementation of GLL Parser in Python including SPPF parse tree extraction [1]. The Python interpreter is embedded so that you can work through the implementation steps.

A GLL parser is a generalization of LL parsers. The first generalized LL parser was reported by Grune and Jacob [2] (11.2) from a masters thesis report in 1993 (another possibly earlier paper looking at generalized LL parsing is from Lang in 1974 [3] and another from Bouckaert et al. [4]). However, a better known generalization of LL parsing was described by Scott and Johnstone [5]. This post follows the later parsing technique. In this post, I provide a complete implementation and a tutorial on how to implement a GLL parser in Python.

We previously discussed Earley parser which is a general context-free parser. GLL parser is another general context-free parser that is capable of parsing strings that conform to **any** given context-free grammar. The algorithm is a generalization of the traditional recursive descent parsing style. In traditional recursive descent parsing, the programmer uses the call stack for keeping track of the parse context. This approach, however, fails when there is left recursion. The problem is that recursive descent parsers cannot advance the parsed index as it is not immediately clear how many recursions are required to parse a given string. Bounding of recursion as we discussed before is a reasonable solution. However, it is very inefficient.

GLL parsing offers a solution. The basic idea behind GLL parsing is to maintain the call stack programmatically, which allows us to iteratively deepen the parse for any nonterminal at any given point. This combined with sharing of the stack (GSS) and generation of parse forest (SPPF) makes the GLL parsing very efficient. Furthermore, unlike Earley, CYK, and GLR parsers, GLL parser operates by producing a custom parser for a given grammar. This means that one can actually debug the recursive descent parsing program directly. Hence, using GLL can be much more friendly to the practitioner.

Similar to Earley, GLR, CYK, and other general context-free parsers, the worst case for parsing is $O(n^3)$. However, for LL(1) grammars, the parse time is $O(n)$.

Synopsis

```
import gllparser as P
my_grammar = {'<start>': [['1', '<A>'],
                          ['2']
                         ],
              '<A>'    : [['a']]}
my_parser = P.compile_grammar(my_grammar)
for tree in my_parser.parse_on(text='1a', start_symbol='<start>'):
    print(P.format_parsetree(tree))
```

Contents

Important: Pyodide takes time to initialize. Initialization completion is indicated by a red border around *Run all* button.

Run all

Definitions

For this post, we use the following terms:

- The *alphabet* is the set all of symbols in the input language. For example, in this post, we use all ASCII characters as alphabet.
- A *terminal* is a single alphabet symbol. Note that this is slightly different from usual definitions (done here for ease of parsing). (Usually a terminal is a contiguous sequence of symbols from the alphabet. However, both kinds of grammars have a one to one correspondence, and can be converted easily.)

  For example, `x` is a terminal symbol.
- A *nonterminal* is a symbol outside the alphabet whose expansion is *defined* in the grammar using *rules* for expansion.

  For example, `<term>` is a nonterminal in the below grammar.
- A *rule* is a finite sequence of *terms* (two types of terms: terminals and nonterminals) that describe an expansion of a given terminal. A rule is also called an *alternative* expansion.

  For example, `[<term>+<expr>]` is one of the expansion rules of the nonterminal `<expr>`.
- A *definition* is a set of *rules* that describe the expansion of a given nonterminal.

  For example, `[[<digit>,<digits>],[<digit>]]` is the definition of the nonterminal `<digits>`
- A *context-free grammar* is composed of a set of nonterminals and corresponding definitions that define the structure of the nonterminal.

  The grammar given below is an example context-free grammar.
- A terminal *derives* a string if the string contains only the symbols in the terminal. A nonterminal derives a string if the corresponding definition derives the string. A definition derives the string if one of the rules in the definition derives the string. A rule derives a string if the sequence of terms that make up the rule can derive the string, deriving one substring after another contiguously (also called parsing).
- A *derivation tree* is an ordered tree that describes how an input string is derived by the given

start symbol. Also called a *parse tree*.
- A derivation tree can be collapsed into its string equivalent. Such a string can be parsed again by the nonterminal at the root node of the derivation tree such that at least one of the resulting derivation trees would be the same as the one we started with.
- The *yield* of a tree is the string resulting from collapsing that tree.
- An *epsilon* rule matches an empty string.

Prerequisites

As before, we start with the prerequisite imports. If you are running this on command line, please uncomment the following line.

```
def __canvas__(g):
   print(g)
```

▶ Available Packages

We need the fuzzer to generate inputs to parse and also to provide some utilities

```
import simplefuzzer as fuzzer
```

Run

We use the `display_tree()` method in earley parser for displaying trees.

```
import earleyparser as ep
```

Run

We use the random choice to extract derivation trees from the parse forest.

```
import random
```

Run

Pydot is needed for drawing

```
import pydot
```

Run

As before, we use the fuzzingbook grammar style. Here is an example grammar for arithmetic expressions, starting at `<start>`. A terminal symbol has exactly one character (Note that we

disallow empty string (`''`) as a terminal symbol). Secondly, as per traditional implementations, there can only be one expansion rule for the `<start>` symbol. We work around this restriction by simply constructing as many charts as there are expansion rules, and returning all parse trees.

Traditional Recursive Descent

Consider how you will parse a string that conforms to the following grammar

```
g1 = {
    '<S>': [
           ['<A>', '<B>'],
           ['<C>']],
   '<A>': [
           ['a']],
   '<B>': [
           ['b']],
   '<C>': [
           ['c']],
}
g1_start = '<S>'
```

Run

In traditional recursive descent, we write a parser in the following fashion

```
class G1TraditionalRD(ep.Parser):
    def recognize_on(self, text):
        res =  self.S(text, 0)
        if res == len(text): return True
        return False

    # S ::= S_0 | S_1
    def S(self, text, cur_idx):
        if (i:= self.S_0(text, cur_idx)) is not None: return i
        if (i := self.S_1(text, cur_idx)) is not None: return i
        return None

    # S_0 ::= <A> <B>
    def S_0(self, text, cur_idx):
        if (i := self.A(text, cur_idx)) is None: return None
        if (i := self.B(text, i)) is None: return None
        return i

    # S_1 ::= <C>
    def S_1(self, text, cur_idx):
        if (i := self.C(text, cur_idx)) is None: return None
        return i

    def A(self, text, cur_idx):
        if (i := self.A_0(text, cur_idx)) is not None: return i
        return None

    # A_0 ::= a
    def A_0(self, text, cur_idx):
        i = cur_idx+1
        if text[cur_idx:i] != 'a': return None
        return i

    def B(self, text, cur_idx):
        if (i := self.B_0(text, cur_idx)) is not None: return i
        return None
```

```
    # B_0 ::= b
    def B_0(self, text, cur_idx):
        i = cur_idx+1
        if text[cur_idx:i] != 'b': return None
        return i

    def C(self, text, cur_idx):
        if (i := self.C_0(text, cur_idx)) is not None: return i
        return None

    # C_0 ::= c
    def C_0(self, text, cur_idx):
        i = cur_idx+1
        if text[cur_idx:i] != 'c': return None
        return i
```

Run

Using it

```
p = G1TraditionalRD()
assert p.recognize_on('ab')
assert p.recognize_on('c')
assert not p.recognize_on('abc')
assert not p.recognize_on('ac')
assert not p.recognize_on('')
```

Run

What if there is recursion? Here is another grammar with recursion

```
g2 = {
    '<S>': [
            ['<A>']],
   '<A>': [
        ['a', '<A>'],
        []]
}
g2_start = '<S>'
```

Run

In traditional recursive descent, we write a parser in the following fashion

```
class G2TraditionalRD(ep.Parser):
    def recognize_on(self, text):
        res =  self.S(text, 0)
        if res == len(text): return True
        return False

    # S ::= S_0
    def S(self, text, cur_idx):
        if (i:= self.S_0(text, cur_idx)) is not None: return i
        return None

    # S_0 ::= <A>
```

```
    def S_0(self, text, cur_idx):
        if (i := self.A(text, cur_idx)) is None: return None
        return i

    def A(self, text, cur_idx):
        if (i := self.A_0(text, cur_idx)) is not None: return i
        if (i := self.A_1(text, cur_idx)) is not None: return i
        return None

    # A_0 ::= a <A>
    def A_0(self, text, cur_idx):
        i = cur_idx+1
        if text[cur_idx:i] != 'a': return None
        if (i := self.A(text, i)) is None: return None
        return i

    # A_1 ::=
    def A_1(self, text, cur_idx):
        return cur_idx
```

Run

Using it

```
p = G2TraditionalRD()
assert p.recognize_on('a')
assert not p.recognize_on('b')
assert p.recognize_on('aa')
assert not p.recognize_on('ab')
assert p.recognize_on('')
```

Run

The problem happens when there is a left recursion. For example, the following grammar contains a left recursion even though it recognizes the same language as before.

```
g3 = {
    '<S>': [
            ['<A>']],
   '<A>': [
        ['<A>', 'a'],
        []]
}
g3_start = '<S>'
```

Run

Naive Threaded Recognizer

The problem with left recursion is that in traditional recursive descent style, we are forced to follow a depth first exploration, completing the parse of one entire rule before attempting then next rule. We can work around this by managing the call stack ourselves. The idea is to convert each procedure into a case label, save the previous label in the stack (managed by us) before a sub procedure. When

the exploration is finished, we pop the previous label off the stack, and continue where we left off.

```
class NaiveThreadedRecognizer(ep.Parser):
    def recognize_on(self, text, start_symbol, max_count=1000):
        parser = self.parser
        parser.initialize(text)
        parser.set_grammar(
        {
         '<S>': [['<A>']],
         '<A>': [['<A>', 'a'],
                 []]
        })
        L, stack_top, cur_idx = start_symbol, parser.stack_bottom, 0
        self.count = 0
        while self.count < max_count:
            self.count += 1
            if L == 'L0':
                if parser.threads:
                    (L, stack_top, cur_idx) = parser.next_thread()
                    if ((L[0], stack_top, cur_idx)
                                    == (start_symbol, parser.stack_bottom,
(parser.m-1))):
                        return parser
                    continue
                else:
                    return []
            elif L == 'L_':
                stack_top = parser.fn_return(stack_top, cur_idx) # pop
                L = 'L0' # goto L_0
                continue

            elif L == '<S>':
                # <S>::=['<A>']
                parser.add_thread( ('<S>',0,0), stack_top, cur_idx)
                L = 'L0'
                continue

            elif L ==  ('<S>',0,0): # <S>::= | <A>
                 stack_top = parser.register_return(('<S>',0,1), stack_top,
cur_idx)
                L = '<A>'
                continue

            elif L ==  ('<S>',0,1): # <S>::= <A> |
                L = 'L_'
                continue

            elif L == '<A>':
                # <A>::=['<A>', 'a']
                parser.add_thread( ('<A>',0,0), stack_top, cur_idx)
                # <A>::=[]
                parser.add_thread( ('<A>',1,0), stack_top, cur_idx)
                L = 'L0'
                continue

            elif L == ('<A>',0,0): # <A>::= | <A> a
                 stack_top = parser.register_return(('<A>',0,1), stack_top,
cur_idx)
                L = "<A>"
                continue

            elif L == ('<A>',0,1): # <A>::= <A> | a
                if parser.I[cur_idx] == 'a':
                    cur_idx = cur_idx+1
                    L = ('<A>',0,2)
                else:
                    L = 'L0'
                continue
```

```
            elif L == ('<A>',0,2): # <A>::= <A> a |
                L = 'L_'
                continue

            elif L == ('<A>',1,0): # <A>::= |
                L = 'L_'
                continue

            else:
                assert False
```

Run

We also need a way to hold the call stack. The call stack is actually stored as a linked list with the current stack_top on the top. With multiple alternatives being explored together, we actually have a tree, but the leaf nodes only know about their parent (not the reverse). For convenience, we use a wrapper for the call-stack, where we define a few book keeping functions. First the initialization of the call stack.

```
class CallStack:
    def initialize(self, s):
        self.threads = []
        self.I = s + '$'
        self.m = len(self.I)
        self.stack_bottom = {'label':('L0', 0), 'previous': []}

    def set_grammar(self, g):
        self.grammar = g
```

Run

Adding a thread simply means appending the label, current stack top, and current parse index to the threads. We can also retrieve threads.

```
class CallStack(CallStack):
    def add_thread(self, L, stack_top, cur_idx):
        self.threads.append((L, stack_top, cur_idx))

    def next_thread(self):
        t, *self.threads = self.threads
        return t
```

Run

Next, we define how returns are handed. That is, before exploring a new sub procedure, we have to save the return label in the stack, which is handled by `register_return()`. The current stack top is added as a child of the return label.

```
class CallStack(CallStack):
    def register_return(self, L, stack_top, cur_idx):
        v = {'label': (L, cur_idx), 'previous': [stack_top]}
        return v
```

Run

When we have finished exploring a given procedure, we return back to the original position in the stack by poping off the prvious label.

```
class CallStack(CallStack):
    def fn_return(self, stack_top, cur_idx):
        if stack_top != self.stack_bottom:
            (L, _k) = stack_top['label']
            for c_st in stack_top['previous']: # only one previous
                self.add_thread(L, c_st, cur_idx)
        return stack_top
```

Run

Using it.

```
p = NaiveThreadedRecognizer()
p.parser = CallStack()
assert p.recognize_on('', '<S>')
print(p.count)
assert p.recognize_on('a', '<S>')
print(p.count)
assert p.recognize_on('aa', '<S>')
print(p.count)
assert p.recognize_on('aaa', '<S>')
print(p.count)
```

Run

This unfortunately has a problem. The issue is that, when a string does not parse, the recursion along with the epsilon rule means that there is always a thread that keeps spawning new threads.

```
assert not p.recognize_on('ab', '<S>', max_count=1000)
print(p.count)
```

Run

The GSS Graph

The way to solve it is to use something called a *graph-structured stack* [6]. A naive conversion of recursive descent parsing to generalized recursive descent parsing can be done by maintaining independent stacks for each thread. However, this approach is has problems as we saw previously, when it comes to left recursion. The GSS converts the tree structured stack to a graph.

The GSS Node

A GSS node is simply a node that can contain any number of children. Each child is actually an edge in the graph.

(Each GSS Node is of the form L_i^j where j is the index of the character consumed. However, we do not need to know the internals of the label here).

```
class GSSNode:
    def __init__(self, label): self.label, self.children = label, []
    def __eq__(self, other): return self.label == other.label
    def __repr__(self): return str((self.label, self.children))
```

Run

The GSS container

Next, we define the graph container. We keep two structures. `self.graph` which is the shared stack, and `self.P` which is the set of labels that went through a `fn_return`, i.e. `pop` operation.

```
class GSS:
    def __init__(self): self.graph, self.P = {}, {}

    def get(self, my_label):
        if my_label not in self.graph:
            self.graph[my_label], self.P[my_label] = GSSNode(my_label), []
        return self.graph[my_label]

    def add_parsed_index(self, label, j):
        self.P[label].append(j)

    def parsed_indexes(self, label):
        return self.P[label]

    def __repr__(self): return str(self.graph)
```

Run

A wrapper for book keeping functions.

```
class GLLStructuredStack:
    def initialize(self, input_str):
        self.I = input_str + '$'
        self.m = len(self.I)
        self.gss = GSS()
        self.stack_bottom = self.gss.get(('L0', 0))
        self.threads = []
        self.U = [[] for j in range(self.m+1)]

    def set_grammar(self, g):
        self.grammar = g
```

Run

GLL+GSS add_thread (add)

Our add_thread increases a bit in complexity. We now check if a thread already exists before starting a new thread.

```
class GLLStructuredStack(GLLStructuredStack):
    def add_thread(self, L, stack_top, cur_idx):
        if (L, stack_top) not in self.U[cur_idx]:  # added
            self.U[cur_idx].append((L, stack_top)) # added
            self.threads.append((L, stack_top, cur_idx))
```

Run

next_thread is same as before

```
class GLLStructuredStack(GLLStructuredStack):
    def next_thread(self):
        t, *self.threads = self.threads
        return t
```

Run

GLL+GSS register_return (create)

A major change in this method. We now look for pre-existing edges before appending edges (child nodes).

```
class GLLStructuredStack(GLLStructuredStack):
    def register_return(self, L, stack_top, cur_idx):
        v = self.gss.get((L, cur_idx))   # added
        v_to_u = [c for c in v.children  # added
                            if c.label == stack_top.label]
        if not v_to_u:                   # added
            v.children.append(stack_top) # added

            for h_idx in self.gss.parsed_indexes(v.label): # added
                self.add_thread(L, stack_top, h_idx)       # added
        return v
```

Run

GLL+GSS fn_return (pop)

A small change in fn_return. We now save all parsed indexes at every label when the parse is complete.

```
class GLLStructuredStack(GLLStructuredStack):
    def fn_return(self, stack_top, cur_idx):
        if stack_top != self.stack_bottom:
            (L, _k) = stack_top.label
            self.gss.add_parsed_index(stack_top.label, cur_idx) # added
            for c_st in stack_top.children: # changed
                self.add_thread(L, c_st, cur_idx)
        return stack_top
```

Run

With GSS, we finally have a true GLL recognizer. Here is the same recognizer unmodified, except for checking the parse ending. Here, we check whether the start symbol is completely parsed only when the threads are complete.

```
class GLLG1Recognizer(ep.Parser):
    def recognize_on(self, text, start_symbol):
        parser = self.parser
        parser.initialize(text)
        parser.set_grammar(
        {
         '<S>': [['<A>']],
         '<A>': [['<A>', 'a'],
                 []]
        })
        L, stack_top, cur_idx = start_symbol, parser.stack_bottom, 0
        while True:
            if L == 'L0':
                if parser.threads:
                    (L, stack_top, cur_idx) = parser.next_thread()
                    continue
                else: # changed
                                                    for   n_alt,   rule   in
enumerate(self.parser.grammar[start_symbol]):
                                    if  ( ((start_symbol, n_alt, len(rule)),
parser.stack_bottom)
                                in parser.U[parser.m-1]):
                                parser.root = (start_symbol, 0, parser.m)
                                return parser
                    return []
            elif L == 'L_':
                stack_top = parser.fn_return(stack_top, cur_idx) # pop
                L = 'L0' # goto L_0
                continue

            elif L == '<S>':
                # <S>::=['<A>']
                parser.add_thread( ('<S>',0,0), stack_top, cur_idx)
                L = 'L0'
                continue

            elif L ==  ('<S>',0,0): # <S>::= | <A>
                 stack_top = parser.register_return(('<S>',0,1), stack_top,
cur_idx)
                L = '<A>'
                continue

            elif L ==  ('<S>',0,1): # <S>::= <A> |
                L = 'L_'
                continue

            elif L == '<A>':
                # <A>::=['<A>', 'a']
                parser.add_thread( ('<A>',0,0), stack_top, cur_idx)
                # <A>::=[]
                parser.add_thread( ('<A>',1,0), stack_top, cur_idx)
                L = 'L0'
                continue

            elif L == ('<A>',0,0): # <A>::= | <A> a
                 stack_top = parser.register_return(('<A>',0,1), stack_top,
cur_idx)
```

```
            L = "<A>"
            continue

        elif L == ('<A>',0,1): # <A>::= <A> | a
            if parser.I[cur_idx] == 'a':
                cur_idx = cur_idx+1
                L = ('<A>',0,2)
            else:
                L = 'L0'
            continue

        elif L == ('<A>',0,2): # <A>::= <A> a |
            L = 'L_'
            continue

        elif L == ('<A>',1,0): # <A>::= |
            L = 'L_'
            continue

        else:
            assert False
```

Run

Using it.

```
p = GLLG1Recognizer()
p.parser = GLLStructuredStack()
assert p.recognize_on('', '<S>')
assert p.recognize_on('a', '<S>')
assert p.recognize_on('aa', '<S>')
assert p.recognize_on('aaa', '<S>')
assert not p.recognize_on('ab', '<S>')
assert not p.recognize_on('aaab', '<S>')
assert not p.recognize_on('baaa', '<S>')
```

Run

GLL Parser

A recognizer is of limited utility. We need the parse tree if we are to use it in practice. Hence, We will now see how to convert this recognizer to a parser.

Utilities.

We start with a few utilities.

Symbols in the grammar

Here, we extract all terminal and nonterminal symbols in the grammar.

```
def symbols(grammar):
    terminals, nonterminals = [], []
    for k in grammar:
        for r in grammar[k]:
            for t in r:
```

```
                if fuzzer.is_nonterminal(t):
                    nonterminals.append(t)
                else:
                    terminals.append(t)
    return (sorted(list(set(terminals))), sorted(list(set(nonterminals))))
```

Run

Using it

```
print(symbols(g1))
```

Run

First, Follow, Nullable sets

To optimize GLL parsing, we need the First, Follow, and Nullable sets. (*Note* we do not use this at present)

Here is a nullable grammar.

```
nullable_grammar = {
    '<start>': [['<A>', '<B>']],
    '<A>': [['a'], [], ['<C>']],
    '<B>': [['b']],
    '<C>': [['<A>'], ['<B>']]
}
```

Run

The definition is as follows.

```
def union(a, b):
    n = len(a)
    a |= b
    return len(a) != n

def get_first_and_follow(grammar):
    terminals, nonterminals = symbols(grammar)
    first = {i: set() for i in nonterminals}
    first.update((i, {i}) for i in terminals)
    follow = {i: set() for i in nonterminals}
    nullable = set()
    while True:
        added = 0
            productions = [(k,rule) for k in nonterminals for rule in
grammar[k]]
        for k, rule in productions:
            can_be_empty = True
            for t in rule:
                added += union(first[k], first[t])
                if t not in nullable:
                    can_be_empty = False
                    break
```

```
            if can_be_empty:
                added += union(nullable, {k})

            follow_ = follow[k]
            for t in reversed(rule):
                if t in follow:
                    added += union(follow[t], follow_)
                if t in nullable:
                    follow_ = follow_.union(first[t])
                else:
                    follow_ = first[t]
        if not added:
            return first, follow, nullable
```

Run

Using

```
first, follow, nullable = get_first_and_follow(nullable_grammar)
print("first:", first)
print("follow:", follow)
print("nullable", nullable)
```

Run

First of a rule fragment.

(*Note* we do not use this at present) We need to compute the expected `first` character of a rule suffix.

```
def get_rule_suffix_first(rule, dot, first, follow, nullable):
    alpha, beta = rule[:dot], rule[dot:]
    fst = []
    for t in beta:
        if fuzzer.is_terminal(t):
            fst.append(t)
            break
        else:
            fst.extend(first[t])
            if t not in nullable: break
            else: continue
    return sorted(list(set(fst)))
```

Run

To verify, we define an expression grammar.

```
grammar = {
    '<start>': [['<expr>']],
    '<expr>': [
        ['<term>', '+', '<expr>'],
        ['<term>', '-', '<expr>'],
        ['<term>']],
    '<term>': [
        ['<fact>', '*', '<term>'],
```

```
        ['<fact>', '/', '<term>'],
        ['<fact>']],
    '<fact>': [
        ['<digits>'],
        ['(','<expr>',')']],
    '<digits>': [
        ['<digit>','<digits>'],
        ['<digit>']],
    '<digit>': [["%s" % str(i)] for i in range(10)],
}

grammar_start = '<start>'
```

Run

using

```
rule_first = get_rule_suffix_first(grammar['<term>'][1], 1, first, follow,
nullable)
print(rule_first)
```

Run

SPPF Graph

We use a data-structure called *Shared Packed Parse Forest* to represent the parse forest. We cannot simply use a parse tree because there may be multiple possible derivations of the same input string (possibly even an infinite number of them). The basic idea here is that multiple derivations (even an infinite number of derivations) can be represented as links in the graph.

The SPPF graph contains four kinds of nodes. The *dummy* node represents an empty node, and is the simplest. The *symbol* node represents the parse of a nonterminal symbol within a given extent (i, j). Since there can be multiple derivations for a nonterminal symbol, each derivation is represented by a *packed* node, which is the third kind of node. Another kind of node is the *intermediate* node. An intermediate node represents a partially parsed rule, containing a prefix rule and a suffix rule. As in the case of symbol nodes, there can be many derivations for a rule fragment. Hence, an intermediate node can also contain multiple packed nodes. A packed node in turn can contain symbol, intermediate, or dummy nodes.

SPPF Node

```
N_ID = 0

class SPPFNode:
    def __init__(self):
        self.children, self.label= [], '<None>'
        self.set_nid()

    def set_nid(self):
        global N_ID
        N_ID += 1
        self.nid  = N_ID
```

```
    def __eq__(self, o): return self.label == o.label

    def add_child(self, child): self.children.append(child)

    def to_s(self, g): return self.label[0]

    def __repr__(self): return 'SPPF:%s' % str(self.label)

    def to_tree(self, hmap, tab): raise NotImplemented

    def to_tree_(self, hmap, tab):
        key = self.label[0] # ignored
        ret = []
        for n in self.children:
            v = n.to_tree_(hmap, tab+1)
            ret.extend(v)
        return ret
```

Run

SPPF Dummy Node

The dummy SPPF node is used to indicate the empty node at the end of rules.

```
class SPPF_dummy_node(SPPFNode):
    def __init__(self, s, i, j):
        self.label, self.children = (s, i, j), []
        self.set_nid()
    def __repr__(self): return 'SPPFdummy:%s' % str(self.label)
```

Run

SPPF Symbol Node

j and i are the extents. Each symbol can contain multiple packed nodes each representing a different derivation. See getNodeP **Note.** In the presence of ambiguous parsing, we choose a derivation at random. So, run the `to_tree()` multiple times to get all parse trees. If you want a better solution, see the forest generation in earley parser which can be adapted here too.

```
class SPPF_symbol_node(SPPFNode):
    def __repr__(self): return 'SPPFsymbol:%s' % str(self.label)
    def __init__(self, x, i, j):
        self.label, self.children = (x, i, j), []
        self.set_nid()

    def to_tree(self, hmap, tab): return self.to_tree_(hmap, tab)[0]
    def to_tree_(self, hmap, tab):
        key = self.label[0]
        if self.children:
            n = random.choice(self.children)
            return [[key, n.to_tree_(hmap, tab+1)]]
        return [[key, []]]
```

Run

SPPF Intermediate Node

Has only two children max (or 1 child).

```
class SPPF_intermediate_node(SPPFNode):
    def __repr__(self): return 'SPPFintermediate:%s' % str(self.label)
    def __init__(self, t, j, i):
        self.label, self.children = (t, j, i), []
        self.set_nid()
```

Run

SPPF Packed Node

```
class SPPF_packed_node(SPPFNode):
    def __repr__(self): return 'SPPFpacked:%s' % str(self.label)
    def __init__(self, t, k):
        self.label, self.children = (t,k), []
        self.set_nid()
```

Run

The GLL parser

We can now build our GLL parser. All procedures change to include SPPF nodes. We first define our initialization

```
class GLLStructuredStackP:
    def initialize(self, input_str):
        self.I = input_str + '$'
        self.m = len(input_str)
        self.gss = GSS()
        self.stack_bottom = self.gss.get(('L0', 0))
        self.threads = []
           self.U = [[] for j in range(self.m+1)] # descriptors for each
index
        self.SPPF_nodes = {}

    def to_tree(self):
        return self.SPPF_nodes[self.root].to_tree(self.SPPF_nodes, tab=0)

    def set_grammar(self, g):
        self.grammar = g
        self.first, self.follow, self.nullable = get_first_and_follow(g)
```

Run

GLL+GSS+SPPF add_thread (add)

From parse tree generation [1] we have:

```
add(L, u, i, w) {
   if ((L, u, w)/∈ Ui { add (L, u, w) to Ui, add (L, u, i, w) to R } }
```

```
class GLLStructuredStackP(GLLStructuredStackP):
    def add_thread(self, L, stack_top, cur_idx, sppf_w):
        if (L, stack_top, sppf_w) not in self.U[cur_idx]:
            self.U[cur_idx].append((L, stack_top, sppf_w))
            self.threads.append((L, stack_top, cur_idx, sppf_w))
```

Run

GLL+GSS+SPPF fn_return (pop)

From parse tree generation [1] we have:

```
pop(u, i, z) {
  if (u/= u0) {
      let (L, k) be the label of u
      add (u, z) to P
      for each edge (u, w, v) {
          let y be the node returned by getNodeP(L, w, z)
          add(L, v, i, y)) } } }
```

in the above, an edge is from u to v labeled w.

```
class GLLStructuredStackP(GLLStructuredStackP):
    def fn_return(self, stack_top, cur_idx, sppf_z):
        if stack_top != self.stack_bottom:
            (L, _k) = stack_top.label
            self.gss.add_parsed_index(stack_top.label, sppf_z)
            for c_st,sppf_w in stack_top.children:
                sppf_y = self.getNodeP(L, sppf_w, sppf_z)
                self.add_thread(L, c_st, cur_idx, sppf_y)
        return stack_top
```

Run

GLL+GSS+SPPF register_return (create)

From parse tree generation [1] we have:

```
create(L, u, i, w) {
    if there is not already a GSS node labelled (L, i) create one
    let v be the GSS node labelled (L, i)
    if there is not an edge from v to u labelled w {
       create an edge from v to u labelled w
       for all ((v, z) ∈ P ) {
          let y be the node returned by getNodeP(L, w, z)
          add(L, u, h, y) where h is the right extent of z } }
    return v }
```

```
class GLLStructuredStackP(GLLStructuredStackP):
    def register_return(self, L, stack_top, cur_idx, sppf_w):
        v = self.gss.get((L, cur_idx))
        v_to_u_labeled_w = [c for c,lbl in v.children
                                    if c.label == stack_top.label and lbl ==
sppf_w]
        if not v_to_u_labeled_w:
            v.children.append((stack_top, sppf_w))

            for sppf_z in self.gss.parsed_indexes(v.label):
                sppf_y = self.getNodeP(L, sppf_w, sppf_z)
                h_idx = sppf_z.label[-1]
                self.add_thread(L, stack_top, h_idx, sppf_y)
        return v
```

Run

GLL+GSS+SPPF utilities.

```
class GLLStructuredStackP(GLLStructuredStackP):
    def next_thread(self):
        t, *self.threads = self.threads
        return t

    def sppf_find_or_create(self, label, j, i):
        n = (label, j, i)
        if  n not in self.SPPF_nodes:
            node = None
            if label is None or label == '$': node = SPPF_dummy_node(*n)
            elif isinstance(label, str): node = SPPF_symbol_node(*n)
            else:                        node = SPPF_intermediate_node(*n)
            self.SPPF_nodes[n] = node
        return self.SPPF_nodes[n]
```

Run

We also need the to produce SPPF nodes correctly.

`getNodeT(x, i)` creates and returns an SPPF node labeled `(x, i, i+1)` or `(epsilon, i, i)` if x is epsilon

From parse tree generation [1] we have:

```
getNodeT (x, i) {
  if (x = ε) h := i else h := i + 1
  if there is no SPPF node labelled (x, i, h) create one
  return the SPPF node labelled (x, i, h) }
```

```
class GLLStructuredStackP(GLLStructuredStackP):
    def getNodeT(self, x, i):
        j = i if x is None else i+1
        return self.sppf_find_or_create(x, i, j)
```

Run

`getNodeP(X::= alpha.beta, w, z)` takes a grammar slot `X ::= alpha . beta` and two SPPF nodes w, and z (z may be dummy node $). the nodes w and z are not packed nodes, and will have labels of form `(s, j, k)` and `(r, k, i)`

From parse tree generation [1] we have:

```
getNodeP(X ::= α · β, w, z) {
  if (α is a terminal or a non-nullable nontermial and if β/= ε) return z
  else { if (β = ε) t := X else t := (X ::= α · β)
  suppose that z has label (q, k, i)
  if (w/= $) { suppose that w has label (s, j, k)
  if there does not exist an SPPF node y labelled (t, j, i) create one
  if y does not have a child labelled (X ::= α · β, k)
           create one with left child w and right child z }
  else {
     if there does not exist an SPPF node y labelled (t, k, i) create one
     if y does not have a child labelled (X ::= α · β, k)
       create one with child z }
  return y } }
```

```python
class GLLStructuredStackP(GLLStructuredStackP):
    def getNodeP(self, X_rule_pos, sppf_w, sppf_z):
        X, nalt, dot = X_rule_pos
        rule = self.grammar[X][nalt]
        alpha, beta = rule[:dot], rule[dot:]

        if self.is_non_nullable_alpha(alpha) and beta: return sppf_z

        t = X if beta == [] else X_rule_pos

        _q, k, i = sppf_z.label
        if self.not_dummy(sppf_w):
            _s,j,_k = sppf_w.label # assert k == _k
            children = [sppf_w,sppf_z]
        else:
            j = k
            children = [sppf_z]

        y = self.sppf_find_or_create(t, j, i)
        if not [c for c in y.children if c.label == (X_rule_pos, k)]:
            pn = SPPF_packed_node(X_rule_pos, k)
            key =  (('P', X_rule_pos, k), j, i)
            assert key not in self.SPPF_nodes
            self.SPPF_nodes[key] = pn
            for c_ in children: pn.add_child(c_)
            y.add_child(pn)
        return y

    def not_dummy(self, sppf_w):
        if sppf_w.label[0] == '$':
            assert isinstance(sppf_w, SPPF_dummy_node)
            return False
        assert not isinstance(sppf_w, SPPF_dummy_node)
        return True
```

```
    def is_non_nullable_alpha(self, alpha):
        if not alpha: return False
        if len(alpha) != 1: return False
        if fuzzer.is_terminal(alpha[0]): return True
        if alpha[0] in self.nullable: return False
        return True
```

Run

We can now use all these to generate trees.

```
class SPPFG1Recognizer(ep.Parser):
    def recognize_on(self, text, start_symbol):
        parser = self.parser
        parser.initialize(text)
        parser.set_grammar(
        {
         '<S>': [['<A>']],
         '<A>': [['<A>', 'a'],
                 []]
        })
        # L contains start nt.
        end_rule = parser.sppf_find_or_create('$', 0, 0)
                L,  stack_top,  cur_idx,  cur_sppf_node  =  start_symbol,
parser.stack_bottom, 0, end_rule
        while True:
            if L == 'L0':
                if parser.threads: # if R != \empty
                                (L,  stack_top,  cur_idx,  cur_sppf_node)  =
parser.next_thread()
                    # goto L
                    continue
                else:
                        # if exists an SPPF node (start_symbol, 0, m) =>
success
                    if (start_symbol, 0, parser.m) in parser.SPPF_nodes:
                          parser.root = (start_symbol, 0, parser.m)
                          return parser
                    else: return []
            elif L == 'L_':
                          stack_top  =  parser.fn_return(stack_top,  cur_idx,
cur_sppf_node) # pop
                L = 'L0' # goto L_0
                continue

            elif L == '<S>':

                # <S>::=['<A>']
                       parser.add_thread(  ('<S>',0,0),  stack_top,  cur_idx,
end_rule)

                L = 'L0'
                continue
            elif L ==  ('<S>',0,0): # <S>::= | <A>

                 stack_top = parser.register_return(('<S>',0,1), stack_top,
cur_idx, cur_sppf_node)
                L = "<A>"
                continue

            elif L == ('<S>',0,1): # <S>::= <A> |
                L = 'L_'
                continue

            elif L == '<A>':
```

```
                # <A>::=['<A>', 'a']
                      parser.add_thread( ('<A>',0,0), stack_top, cur_idx,
end_rule)
                # <A>::=[]
                      parser.add_thread( ('<A>',1,0), stack_top, cur_idx,
end_rule)

                L = 'L0'
                continue
            elif L ==  ('<A>',0,0): # <A>::= | <A> a
                 stack_top = parser.register_return(('<A>',0,1), stack_top,
cur_idx, cur_sppf_node)
                L = "<A>"
                continue

            elif L == ("<A>",0,1): # <A>::= <A> | a
                if parser.I[cur_idx] == 'a':
                     right_sppf_child = parser.getNodeT(parser.I[cur_idx],
cur_idx)
                    cur_idx = cur_idx+1
                    L = ("<A>",0,2)
                        cur_sppf_node = parser.getNodeP(L, cur_sppf_node,
right_sppf_child)
                else:
                    L = 'L0'
                continue

            elif L == ('<A>',0,2): # <A>::= <A> a |
                L = 'L_'
                continue

            elif L == ("<A>", 1, 0): # <A>::= |
                 # epsilon: If epsilon is present, we skip the end of rule
with same
                # L and go directly to L_
                right_sppf_child = parser.getNodeT(None, cur_idx)
                        cur_sppf_node = parser.getNodeP(L, cur_sppf_node,
right_sppf_child)
                L = 'L_'
                continue


            else:
                assert False
```

Run

We need trees

```
class SPPFG1Recognizer(SPPFG1Recognizer):
    def parse_on(self, text, start_symbol):
        p = self.recognize_on(text, start_symbol)
        return [p.to_tree()]
```

Run

Using it

```
p = SPPFG1Recognizer()
p.parser = GLLStructuredStackP()
```

```
for tree in p.parse_on('aa', start_symbol='<S>'):
    print(ep.display_tree(tree))
```

Run

Building the parser with GLL

At this point, we are ready to build our parser compiler.

Compiling an empty rule

We start with compiling an epsilon rule.

```
def compile_epsilon(g, key, n_alt):
    return '''\
        elif L == ("%s", %d, 0): # %s
            # epsilon: If epsilon is present, we skip the end of rule with
same
            # L and go directly to L_
            right_sppf_child = parser.getNodeT(None, cur_idx)
                    cur_sppf_node  =  parser.getNodeP(L,  cur_sppf_node,
right_sppf_child)
            L = 'L_'
            continue
''' % (key, n_alt, ep.show_dot(key, g[key][n_alt], 0))
```

Run

Using it.

```
v = compile_epsilon(grammar, '<expr>', 1)
print(v)
```

Run

Compiling a Terminal Symbol

```
def compile_terminal(g, key, n_alt, r_pos, r_len, token):
    if r_len == r_pos:
        Lnxt = '"L_"'
    else:
        Lnxt = '("%s",%d,%d)' % (key, n_alt, r_pos+1)
    return '''\
        elif L == ("%s",%d,%d): # %s
            if parser.I[cur_idx] == '%s':
                    right_sppf_child = parser.getNodeT(parser.I[cur_idx],
cur_idx)
                cur_idx = cur_idx+1
                L = %s
                        cur_sppf_node  =  parser.getNodeP(L,  cur_sppf_node,
right_sppf_child)
            else:
                L = 'L0'
            continue
```

```
''' % (key, n_alt, r_pos, ep.show_dot(key, g[key][n_alt], r_pos), token,
Lnxt)
```

Run

Compiling a Nonterminal Symbol

```
def compile_nonterminal(g, key, n_alt, r_pos, r_len, token):
    if r_len == r_pos:
        Lnxt = '"L_"'
    else:
        Lnxt = "('%s',%d,%d)" % (key, n_alt, r_pos+1)
    return '''\
        elif L ==  ('%s',%d,%d): # %s
                stack_top = parser.register_return(%s, stack_top, cur_idx,
cur_sppf_node)
            L = "%s"
            continue
''' % (key, n_alt, r_pos, ep.show_dot(key, g[key][n_alt], r_pos), Lnxt,
token)
```

Run

Using it.

```
rule = grammar['<expr>'][1]
for i, t in enumerate(rule):
    if fuzzer.is_nonterminal(t):
        v = compile_nonterminal(grammar, '<expr>', 1, i, len(rule), t)
    else:
        v = compile_terminal(grammar, '<expr>', 1, i, len(rule), t)
    print(v)
```

Run

Compiling a Rule

`n_alt` is the position of `rule`.

```
def compile_rule(g, key, n_alt, rule):
    res = []
    if not rule:
        r = compile_epsilon(g, key, n_alt)
        res.append(r)
    else:
        for i, t in enumerate(rule):
            if fuzzer.is_nonterminal(t):
                r = compile_nonterminal(g, key, n_alt, i, len(rule), t)
            else:
                r = compile_terminal(g, key, n_alt, i, len(rule), t)
            res.append(r)
        # if epsilon present, we do not want this branch.
        res.append('''\
        elif L == ('%s',%d,%d): # %s
            L = 'L_'
```

```
            continue
''' % (key, n_alt, len(rule), ep.show_dot(key, g[key][n_alt], len(rule))))
    return '\n'.join(res)
```

Run

Using it.

```
v = compile_rule(grammar, '<expr>', 1, grammar['<expr>'][1])
print(v)
```

Run

Compiling a Definition

Note that if performance is important, you may want to check if the current input symbol at `parser.I[cur_idx]` is part of the following, where X is a nonterminal and p is a rule fragment. Note that if you care about the performance, you will want to pre-compute first[p] for each rule fragment `rule[j:]` in the grammar, and first and follow sets for each symbol in the grammar. This should be checked before `parser.add_thread`.

```
def test_select(a, X, p, rule_first, follow):
    if a in rule_first[p]: return True
    if '' not in rule_first[p]: return False
    return a in follow[X]
```

Run

Given that removing this check does not affect the correctness of the algorithm, I have chosen not to add it.

```
def compile_def(g, key, definition):
    res = []
    res.append('''\
        elif L == '%s':
''' % key)
    for n_alt,rule in enumerate(definition):
        res.append('''\
            # %s
                    parser.add_thread(  ('%s',%d,0),  stack_top,  cur_idx,
end_rule)''' % (key + '::=' + str(rule), key, n_alt))
    res.append('''
            L = 'L0'
            continue''')
    for n_alt, rule in enumerate(definition):
        r = compile_rule(g, key, n_alt, rule)
        res.append(r)
    return '\n'.join(res)
```

Run

Using it.

```
v = compile_def(grammar, '<expr>', grammar['<expr>'])
print(v)
```

Run

A template.

```
class GLLParser(ep.Parser):
    def recognize_on(self, text, start_symbol):
        raise NotImplemented()
```

Run

Compiling a Grammar

```
def compile_grammar(g, evaluate=True):
    import pprint
    pp = pprint.PrettyPrinter(indent=4)
    res = ['''\
def recognize_on(self, text, start_symbol):
    parser = self.parser
    parser.initialize(text)
    parser.set_grammar(
%s
    )
    # L contains start nt.
    end_rule = parser.sppf_find_or_create('$', 0, 0)
            L,   stack_top,   cur_idx,   cur_sppf_node   =   start_symbol,
parser.stack_bottom, 0, end_rule
    while True:
        if L == 'L0':
            if parser.threads: # if R != \empty
                                (L,  stack_top,  cur_idx,  cur_sppf_node)  =
parser.next_thread()
                # goto L
                continue
            else:
                    # if there is an SPPF node (start_symbol, 0, m) then
report success
                if (start_symbol, 0, parser.m) in parser.SPPF_nodes:
                      parser.root = (start_symbol, 0, parser.m)
                      return parser
                else: return []
        elif L == 'L_':
                      stack_top  =  parser.fn_return(stack_top,  cur_idx,
cur_sppf_node) # pop
            L = 'L0' # goto L_0
            continue
    ''' % pp.pformat(g)]
    for k in g:
        r = compile_def(g, k, g[k])
        res.append(r)
    res.append('''
        else:
            assert False''')
    res.append('''
```

```
def parse_on(self, text, start_symbol):
    p = self.recognize_on(text, start_symbol)
    return [p.to_tree()]
    ''')

    parse_src = '\n'.join(res)
    s = GLLParser()
    s.src = parse_src
    if not evaluate: return parse_src
    l, g = locals().copy(), globals().copy()
    exec(parse_src, g, l)
    s.parser = GLLStructuredStackP()
    s.parse_src = parse_src
    s.recognize_on = l['recognize_on'].__get__(s)
    s.parse_on = l['parse_on'].__get__(s)
    return s
```

Run

Using it

```
v = compile_grammar(grammar, False)
print(v)
```

Run

Running it

```
G1 = {
    '<S>': [['c']]
}
mystring = 'c'
p = compile_grammar(G1)
v = p.parse_on(mystring, '<S>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring
ep.display_tree(v)
```

Run

SPPF Parse Forest

Let us see how to visualize the forest

```
def to_graph(forest):
    forest.SPPF_nodes[forest.root]
    G = pydot.Dot("my_graph", graph_type="digraph")
    for k in forest.SPPF_nodes:
        cnode = forest.SPPF_nodes[k]
        label = "%s :%d" % (cnode, cnode.nid)
        shape = 'rectangle'# rectangle, oval, diamond
        if isinstance(cnode, SPPF_packed_node):
            label = "%d" % (cnode.nid)
            shape = 'oval'
```

```
        elif isinstance(cnode, SPPF_symbol_node):
            label = "%s: %d" % (cnode.label, cnode.nid)
        elif isinstance(cnode, SPPF_intermediate_node):
            label = "%s: %d" % (cnode.label, cnode.nid)
        elif isinstance(cnode, SPPF_dummy_node):
            label = "$: %d" % (cnode.nid)
        else:
            label = "%s: %d" % (cnode, cnode.nid)

        G.add_node(pydot.Node(str(cnode.nid),
            label=label,
            shape=shape,
            peripheries= '2' if k == forest.root else '1'
        ))
        ns = G.get_node(str(cnode.nid))
        for cn in cnode.children:
                        G.add_edge(pydot.Edge(str(cnode.nid),  str(cn.nid),
color='black'))
    return G
```

Run

Testing it

```
G1 = {
    '<S>': [['<A>','<B>'],
            ['<B>']],
    '<A>': [['a']],
    '<B>': [['b']]
}
p = compile_grammar(G1)
mystring = 'ab'
forest = p.recognize_on(mystring, '<S>')
g = to_graph(forest)
```

Run

We can view it as follows:

```
__canvas__(str(g))
```

Run

Extracting all trees

Previously, we examined how to extract a single parse tree. However, this in insufficient in many cases. Given that context-free grammars can contain ambiguity we want to extract all possible parse trees. To do that, we need to keep track of all choices we make.

ChoiceNode

The ChoiceNode is a node in a linked list of choices. The idea is that whenever there is a choice between exploring different derivations, we pick the first candidate, and make a note of that choice.

Then, during further explorations of the child nodes, if more choices are necessary, those choices are marked in nodes linked from the current node.

- `_chosen` contains the current choice
- `next` holds the next choice done using _chosen
- `total` holds he total number of choices for this node.

```
class ChoiceNode:
    def __init__(self, parent, total):
        self._p, self._chosen = parent, 0
        self._total, self.next = total, None

    def __str__(self):
        return '(%s/%s %s)' % (str(self._chosen),
                               str(self._total), str(self.next))

    def __repr__(self): return repr((self._chosen, self._total))
```

Run

The `chosen()` returns the current candidate.

```
class ChoiceNode(ChoiceNode):
    def chosen(self): return self._chosen
```

Run

A ChoiceNode has exhausted its choices if the current candidate chosen does not exist.

```
class ChoiceNode(ChoiceNode):
    def finished(self):
        return self._chosen >= self._total
```

Run

At the end of generation of a single tree, we increment the candidate number in the last node in the linked list. Then, we check if the last node can provide another candidate to explore. If the node has not exhausted its candidates, then we have nothing more to do. However, if the node has exhausted its candidates, we look for possible candidates in its parent.

```
class ChoiceNode(ChoiceNode):
    def increment(self):
        # as soon as we increment, next becomes invalid
        self.next = None
        self._chosen += 1
        if self.finished():
            if self._p is None: return None
            return self._p.increment()
        return self
```

Run

EnhancedExtractor

The EnhancedExtractor classes uses the choice linkedlist to explore possible parses.

```
class EnhancedExtractor:
    def __init__(self, forest):
        self.my_forest = forest
        self.choices = ChoiceNode(None, 1)
```

Run

Whenever there is a choice to be made, we look at the current node in the choices linked list. If a previous iteration has exhausted all candidates, we have nothing left. In that case, we simply return None, and the updated linkedlist

```
class EnhancedExtractor(EnhancedExtractor):
    def choose_path(self, arr, choices):
        arr_len = len(arr)
        if choices.next is not None:
            if choices.next.finished():
                return None, None, None, choices.next
        else:
            choices.next = ChoiceNode(choices, arr_len)
        next_choice = choices.next.chosen()
        return arr[next_choice], next_choice, arr_len, choices.next
```

Run

While extracting, we have a choice. Should we allow infinite forests, or should we have a finite number of trees with no direct recursion? A direct recursion is when there exists a parent node with the same nonterminal that parsed the same span. We choose here not to extract such trees. They can be added back after parsing.

This is a recursive procedure that inspects a node, extracts the path required to complete that node. A single path (corresponding to a nonterminal) may again be composed of a sequence of smaller paths. Such paths are again extracted using another call to extract_a_node() recursively.

What happens when we hit on one of the node recursions we want to avoid? In that case, we return the current choice node, which bubbles up to `extract_a_tree()`. That procedure increments the last choice, which in turn increments up the parents until we reach a choice node that still has options to explore.

What if we hit the end of choices for a particular choice node (i.e, we have exhausted paths that can be taken from a node)? In this case also, we return the current choice node, which bubbles up to `extract_a_tree()`. That procedure increments the last choice, which bubbles up to the next choice that has some unexplored paths.

```
class EnhancedExtractor(EnhancedExtractor):
    def extract_a_node(self, forest_node, seen, choices):
        if isinstance(forest_node, SPPF_dummy_node):
            return ('', []), choices

        elif isinstance(forest_node, SPPF_packed_node):
            key = forest_node.label[0] # ignored
            ret = []
            for n in forest_node.children:
                v, new_choices = self.extract_a_node(n, seen, choices)
                if v is None: return None, new_choices

                choices = new_choices
                if isinstance(n, SPPF_packed_node):
                    assert v[0] is None
                    ret.extend(v[1])
                elif isinstance(n, SPPF_symbol_node):
                    assert v[0] is not None
                    ret.append(v)
                elif isinstance(n, SPPF_intermediate_node):
                    assert v[0] is None
                    ret.extend(v[1])
                elif isinstance(n, SPPF_dummy_node):
                    assert v[0] is None
                    ret.extend(v[1])
                else:
                    assert False
                    ret.append(v)
            return (None, ret), choices

        elif isinstance(forest_node, (SPPF_intermediate_node)):
            if not forest_node.children:
                return (None, []), choices

            cur_path, _i, _l, new_choices = self.choose_path(
                    forest_node.children, choices)

            if cur_path is None: assert False
            if cur_path.nid in seen: return None, new_choices

            key = forest_node.label[0] # ignored

                    v, new_choices = self.extract_a_node(cur_path, seen | {cur_path.nid}, new_choices) # SPPFintermediate:(('<S3>', 0, 2), 0, 3)(23)
            if v is None: return None, new_choices
            key, children = v
            return (None, children), new_choices



        elif isinstance(forest_node, SPPF_symbol_node):
            if not forest_node.children:
                return (forest_node.label[0], []), choices
            cur_path, _i, _l, new_choices = self.choose_path(
                    forest_node.children, choices)
            if cur_path is None:
                return None, new_choices
            if cur_path.nid in seen: return None, new_choices
            n, newer_choices = self.extract_a_node(
                    cur_path, seen | {cur_path.nid}, new_choices)
            if n is None: return None, newer_choices
            (key, children) = n
            assert key is None
            return (forest_node.label[0], children), newer_choices
```

Run

The extract_a_tree extracts one parse tree at a time, and keeps track of the choices.

```
class EnhancedExtractor(EnhancedExtractor):
    def extract_a_tree(self):
        choices = self.choices
        while not self.choices.finished():
            parse_tree, choices = self.extract_a_node(
                    self.my_forest.SPPF_nodes[self.my_forest.root],
                    set(), self.choices)
            choices.increment()
            if parse_tree is not None:
                return parse_tree
        return None
```

Run

Running it

```
G1 = {
    '<S>': [['<A>'],
            ['<B>']],
    '<A>': [['a']],
    '<B>': [['a']]
}
mystring = 'a'
p = compile_grammar(G1)
forest = p.recognize_on(mystring, '<S>')
ee = EnhancedExtractor(forest)
while True:
    t = ee.extract_a_tree()
    if t is None: break
    s = fuzzer.tree_to_string(t)
assert s == mystring
```

Run

A larger example

```
a_grammar = {
'<start>': [['<expr>']],
'<expr>': [
    ['<expr>', '+', '<expr>'],
    ['<expr>', '-', '<expr>'],
    ['<expr>', '*', '<expr>'],
    ['<expr>', '/', '<expr>'],
    ['(', '<expr>', ')'],
    ['<integer>']],
'<integer>': [
    ['<digits>']],
'<digits>': [
    ['<digit>','<digits>'],
    ['<digit>']],
'<digit>': [["%s" % str(i)] for i in range(10)],
}
mystring = '1+2+3+4'
```

```
p = compile_grammar(a_grammar)
forest = p.recognize_on(mystring, grammar_start)
assert forest
ee = EnhancedExtractor(forest)
while True:
    t = ee.extract_a_tree()
    if t is None: break
    s = fuzzer.tree_to_string(t)
    assert s == mystring
    ep.display_tree(t)
```

Run

A few more examples

2

```
G2 = {
    '<S>': [['c', 'c']]
}
mystring = 'cc'
p = compile_grammar(G2)
v = p.parse_on(mystring, '<S>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring
ep.display_tree(v)
```

Run

3

```
G3 = {
    '<S>': [['c', 'c', 'c']]
}
mystring = 'ccc'
p = compile_grammar(G3)
v = p.parse_on(mystring, '<S>')[0]
r = fuzzer.tree_to_string(v)
assert r == mystring
ep.display_tree(v)
```

Run

4

```
G4 = {
    '<S>': [['c'],
            ['a']]
}
mystring = 'a'
p = compile_grammar(G4)
v = p.parse_on(mystring, '<S>')[0]
r = fuzzer.tree_to_string(v)
assert r == mystring
```

```
ep.display_tree(v)
```

Run

5

```
G5 = {
    '<S>': [['<A>']],
    '<A>': [['a']]
}
mystring = 'a'
p = compile_grammar(G5)
v = p.parse_on(mystring, '<S>')[0]
r = fuzzer.tree_to_string(v)
assert r == mystring
ep.display_tree(v)
```

Run

Expression

1

```
mystring = '(1+1)*(23/45)-1'
p = compile_grammar(grammar)
v = p.parse_on(mystring, grammar_start)[0]
r = fuzzer.tree_to_string(v)
assert r == mystring
ep.display_tree(v)
```

Run

2

Since we use a random choice to get different parse trees in ambiguity, click the run button again and again to get different parses.

```
a_grammar = {
'<start>': [['<expr>']],
'<expr>': [
    ['<expr>', '+', '<expr>'],
    ['<expr>', '-', '<expr>'],
    ['<expr>', '*', '<expr>'],
    ['<expr>', '/', '<expr>'],
    ['(', '<expr>', ')'],
    ['<integer>']],
'<integer>': [
    ['<digits>']],
'<digits>': [
    ['<digit>','<digits>'],
    ['<digit>']],
'<digit>': [["%s" % str(i)] for i in range(10)],
}
```

```
mystring = '1+2+3+4'
p = compile_grammar(a_grammar)
v = p.parse_on(mystring, grammar_start)[0]
r = fuzzer.tree_to_string(v)
assert r == mystring
ep.display_tree(v)
```

Run

A few more examples

```
RR_GRAMMAR2 = {
    '<start>': [
    ['b', 'a', 'c'],
    ['b', 'a', 'a'],
    ['b', '<A>', 'c']
    ],
    '<A>': [['a']],
}
mystring = 'bac'
p = compile_grammar(RR_GRAMMAR2)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

RR_GRAMMAR3 = {
    '<start>': [['c', '<A>']],
    '<A>': [['a', 'b', '<A>'], []],
}
mystring = 'cabababababab'

p = compile_grammar(RR_GRAMMAR3)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print(7)

RR_GRAMMAR4 = {
    '<start>': [['<A>', 'c']],
    '<A>': [['a', 'b', '<A>'], []],
}
mystring = 'abababababc'

p = compile_grammar(RR_GRAMMAR4)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print(8)

RR_GRAMMAR5 = {
'<start>': [['<A>']],
'<A>': [['a', 'b', '<B>'], []],
'<B>': [['<A>']],
}
mystring = 'abababab'

p = compile_grammar(RR_GRAMMAR5)
v = p.parse_on(mystring, '<start>')[0]
print(v)
```

```
r = fuzzer.tree_to_string(v)
assert r == mystring

print(9)

RR_GRAMMAR6 = {
'<start>': [['<A>']],
'<A>': [['a', '<B>'], []],
'<B>': [['b', '<A>']],
}
mystring = 'abababab'

p = compile_grammar(RR_GRAMMAR6)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print(10)

RR_GRAMMAR7 = {
'<start>': [['<A>']],
'<A>': [['a', '<A>'], ['a']],
}
mystring = 'aaaaaaaa'

p = compile_grammar(RR_GRAMMAR7)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print(11)

RR_GRAMMAR8 = {
'<start>': [['<A>']],
'<A>': [['a', '<A>'], ['a']]
}
mystring = 'aa'

p = compile_grammar(RR_GRAMMAR8)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print(12)

X_G1 = {
    '<start>': [['a']],
}
mystring = 'a'
p = compile_grammar(X_G1)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print('X_G1')

X_G2 = {
    '<start>': [['a', 'b']],
}
mystring = 'ab'
p = compile_grammar(X_G2)
v = p.parse_on(mystring, '<start>')[0]
print('X_G2')

X_G3 = {
```

```
    '<start>': [['a', '<b>']],
    '<b>': [['b']]
}
mystring = 'ab'
p = compile_grammar(X_G3)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print('X_G3')

X_G4 = {
    '<start>': [
    ['a', '<a>'],
    ['a', '<b>'],
    ['a', '<c>']
    ],
    '<a>': [['b']],
    '<b>': [['b']],
    '<c>': [['b']]
}
mystring = 'ab'
p = compile_grammar(X_G4)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring


print('X_G4')

X_G5 = {
    '<start>': [['<expr>']],
    '<expr>': [
        ['<expr>', '+', '<expr>'],
        ['1']]
}
X_G5_start = '<start>'

mystring = '1+1'
p = compile_grammar(X_G5)
v = p.parse_on(mystring, '<start>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring


print('X_G5')

X_G6 = {
    '<S>': [
    ['b', 'a', 'c'],
    ['b', 'a', 'a'],
    ['b', '<A>', 'c'],
    ],
    '<A>': [
        ['a']]
}
X_G6_start = '<S>'

mystring = 'bac'
p = compile_grammar(X_G6)
v = p.parse_on(mystring, '<S>')[0]
print(v)
r = fuzzer.tree_to_string(v)
assert r == mystring

print('X_G6')
```

Run

We assign format parse tree so that we can refer to it from this module

```
def format_parsetree(t):
    return ep.format_parsetree(t)
```

Run

Note: The bug that was there previously in EnhancedExtractor has now been fixed. The follow was its test case.

```
gamma_2 = { "<S>":
            [['<S3>'],
             ['<S2>'],
             ["x"],],
            '<S3>': [["<Sx>", "<Sx>", "<Sx>"]],
            '<S2>': [["<S>","<S>"]],
            '<Sx>': [['<S2>'], ['<S3>'], ['x']],
            }
p = compile_grammar(gamma_2)
f = p.recognize_on('xxxx', '<S>')
ee = EnhancedExtractor(f)
while True:
    t = ee.extract_a_tree()
    if t is None: break
    s = fuzzer.tree_to_string(t)
    assert s == 'xxxx'
```

Run

Here is a torture test

```
gamma_3 = { "<S>":
            [['<S>', '<S>'],
             ['<S>', '<S>', '<S>'],
             ["x"]],
            }
string = 'xxxxxx'
p = compile_grammar(gamma_3)
f = p.recognize_on(string, '<S>')
ee = EnhancedExtractor(f)
while True:
    t = ee.extract_a_tree()
    if t is None: break
    s = fuzzer.tree_to_string(t)
    assert s == string
print('done')
```

Run

Note: There is now (2024) a reference implementation for GLL from the authors. It is available at https://github.com/AJohnstone2007/referenceImplementation.

Run all

Artifacts

The runnable Python source for this notebook is available here.

The installable python wheel `gllparser` is available here.

1. Elizabeth Scott, Adrian Johnstone. "GLL parse-tree generation." Science of Computer Programming 78.10 (2013): 1828-1844. ↩ ↩2 ↩3 ↩4 ↩5 ↩6
2. Dick Grune and Ceriel J.H. Jacobs "Parsing Techniques A Practical Guide" 2008 ↩
3. Bernard Lang. "Deterministic techniques for efficient non-deterministic parsers." International Colloquium on Automata, Languages, and Programming. Springer, Berlin, Heidelberg, 1974. ↩
4. M. Bouckaert, Alain Pirotte, M. Snelling. "Efficient parsing algorithms for general context-free parsers." Information Sciences 8.1 (1975): 1-26. ↩
5. Elizabeth Scott, Adrian Johnstone. "GLL parsing." Electronic Notes in Theoretical Computer Science 253.7 (2010): 177-189. ↩
6. Masaru Tomita. Efficient parsing for natural language: a fast algorithm for practical systems. Kluwer Academic Publishers, Boston, 1986. ↩